

API Security Risk Analysis Report

Project Title: API Security Risk Analysis Report

Internship Domain: Cyber Security

Submitted by: Surayya Banu K

Program: Cyber Security Internship

Organization: Future Interns

Date:24/08/2026

1. Introduction

APIs (Application Programming Interfaces) are an essential part of modern applications, allowing different software systems to communicate and exchange data. Since APIs often handle important information and provide access to application resources, proper security controls are necessary to prevent unauthorized access and misuse.

This report presents an API Security Risk Analysis of the **JSONPlaceholder public test API**, conducted using **Postman**. The assessment focuses on endpoint access, authentication and authorization, response data, HTTP headers, query parameters, and error handling. The findings are based on the actual Postman requests, responses, and screenshots collected during the testing process.

2. Objective

The main objective of this task is to perform a security risk analysis of the **JSONPlaceholder API** using **Postman**. The analysis aims to examine API endpoints, responses, and security-related headers to identify potential security risks.

The assessment focuses on:

- Checking authentication and access controls.
- Analyzing response headers and security configurations.
- Reviewing CORS and rate-limiting controls.
- Identifying information disclosure and other security weaknesses.
- Assessing the possible impact of the identified risks.
- Recommending suitable security measures to improve API security.

3. Scope and Ethical Boundaries

In Scope

- JSONPlaceholder public test API.

- Safe GET requests using Postman.
- Endpoint and response inspection.
- HTTP header inspection.
- Query-parameter behavior and safe invalid identifiers.

Out of Scope

- Exploitation or authentication bypass.
- Brute-force testing or automated high-volume requests.
- DoS/flooding activity.
- Testing private or production APIs.
- Accessing or manipulating real users' private data.

4. API Selected

API Name: JSONPlaceholder

Base URL: <https://jsonplaceholder.typicode.com>

API Type: Public REST test/demo API

Description: JSONPlaceholder provides sample resources such as users and posts for testing and prototyping. Because it is intentionally public, unauthenticated access by itself must be interpreted as an observed characteristic of the demo API rather than automatically treated as a production vulnerability.

5. Tools Used

- Postman – Used to send API requests and analyze responses, status codes, and HTTP headers.
- JSONPlaceholder – Used as the API testing target.
- Web Browser (Chrome) – Used to access and verify API endpoints when required.
- Microsoft Word / Canva – Used to prepare and format the final report.
- GitHub – Used to upload and submit the project documentation

6. Methodology

The API security analysis was performed using **Postman** on the **JSONPlaceholder public test API**. The following steps were carried out:

- Selected the JSONPlaceholder API and identified the endpoints to be tested.
- Sent GET requests to different endpoints and examined the responses.
- Checked HTTP status codes and response data.
- Tested individual user resources using different user IDs.
- Reviewed response data for possible information exposure.
- Tested query-parameter filtering using userId.
- Inspected HTTP response headers, including security and rate-limiting information.
- Tested a non-existent resource to observe error handling.
- Analyzed the observations and identified relevant security risks.

7. Endpoints Tested

Endpoint	Purpose	Observed Result
GET /users	Retrieve user collection	200 OK; multiple user fields returned
GET /users/1	Retrieve one user	200 OK; individual user object returned
GET /users/2	Retrieve another user	200 OK; individual user object returned
GET /posts/1	Retrieve one post	200 OK; JSON post object returned
GET /posts?userId=1	Filter posts by userId	200 OK; filtered post collection returned
GET /posts/99999	Invalid/nonexistent identifier	404 Not Found; empty JSON object shown

8. Identified Security Risks and Observations

Risk 1: No Authentication Mechanism

Severity: High

Observation:

The API allows access to user and post endpoints without requiring authentication credentials, API keys, or tokens. The /users, /users/1, and /users/2 endpoints returned successful responses without authentication.

Impact:

Unauthenticated users may access API data directly, increasing the risk of unauthorized data exposure.

Remediation:

Implement proper authentication using API keys, OAuth 2.0, JWT, or another appropriate authentication mechanism.

Risk 2: Excessive Data Exposure

Severity: Medium

Observation:

The GET /users response exposes extensive user information, including names, usernames, email addresses, phone numbers, addresses, geographic coordinates, websites, and company information.

Impact:

Unnecessary exposure of user information can create privacy risks and provide attackers with additional information about users.

Remediation:

Return only the data required by the application and avoid exposing unnecessary personal information.

Risk 3: Technology Information Disclosure

Severity: Low

Observation:

The response headers include X-Powered-By: Express, which reveals that the API is using the Express framework.

Impact:

Technology information can help attackers identify the underlying technology stack and potentially research framework-specific vulnerabilities.

Remediation:

Remove or suppress technology-identifying response headers such as X-Powered-By.

Risk 4: Missing/Incomplete Security Headers

Severity: Low to Medium

Observation:

The response headers show X-Content-Type-Options: nosniff, but other commonly recommended security headers are not visible in the captured response, such as Content-Security-Policy and X-Frame-Options.

Impact:

Missing security headers can reduce protection against certain browser-based attacks and unsafe content handling.

Remediation:

Configure appropriate security headers, including Content-Security-Policy and suitable frame-protection controls, according to the application's requirements.

8.1 Positive Security Observations

- **X-Content-Type-Options:** nosniff is present in the response headers.
- **Rate-limit headers** are visible, including limit and remaining-request values.
- The invalid /posts/99999 request returned 404 Not Found with an empty JSON object rather than a stack trace or verbose internal error.
- The assessment remained read-only and did not perform exploitation, bypass or flooding.

9. Business Impact

The technical observations above are most important when translated into business consequences. If similar API behavior occurred in a production system containing real customer or company information, the main risks could include:

- **Unauthorized access:** Lack of authentication may allow unauthorized users to access API resources.
- **Data privacy concerns:** Excessive exposure of user information may lead to privacy and data-protection issues.
- **Increased attack risk:** Technology information disclosed through response headers may help attackers understand the API environment.
- **Reduced security protection:** Missing or incomplete security headers may increase exposure to certain web-based attacks.
- **Reputational damage:** A security incident could result in loss of customer trust and damage to the organization's reputation.

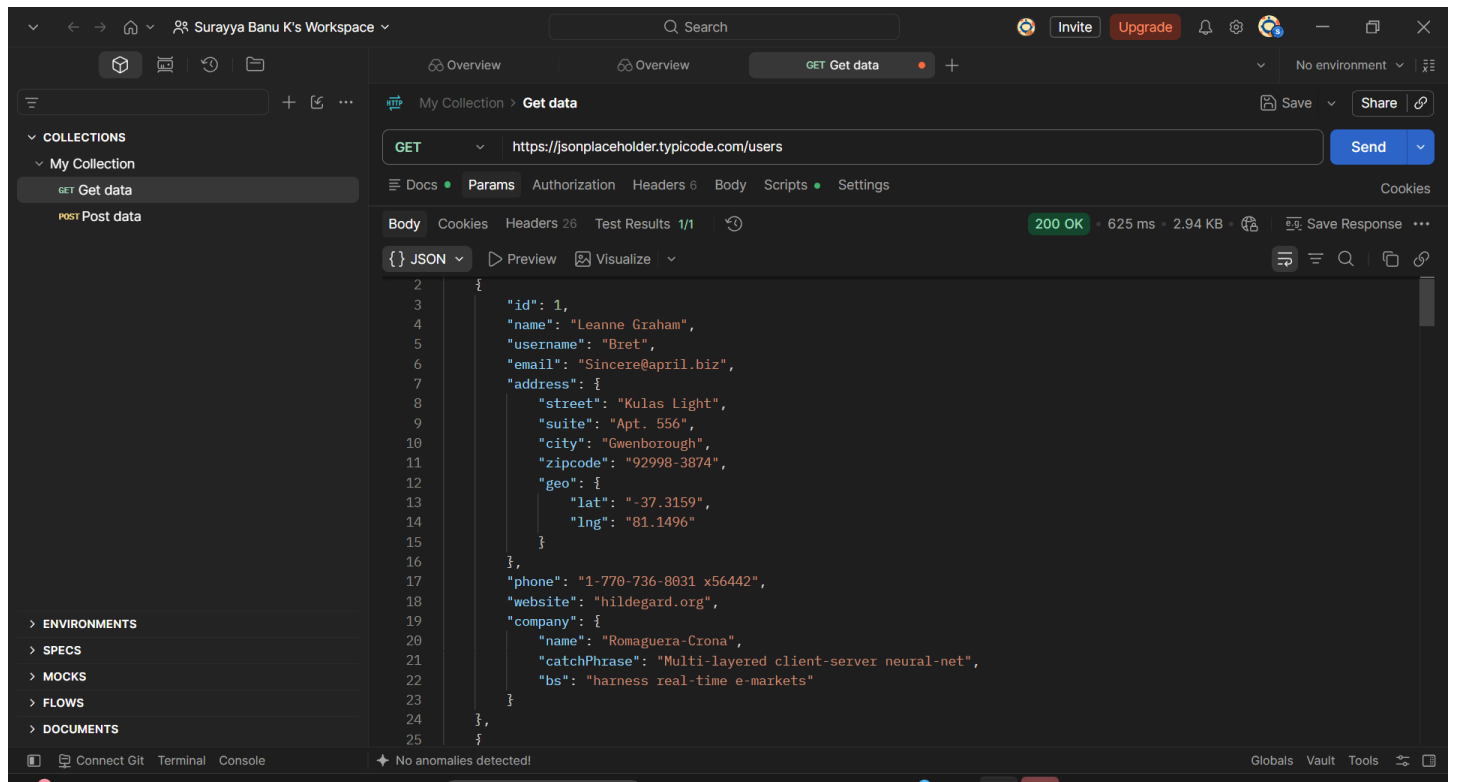
10. Remediation Recommendations

1. **Implement strong authentication** for protected API resources.
2. **Apply data minimization** and return only the information required by the client.
3. **Remove unnecessary technology-identifying headers**, such as X-Powered-By.
4. **Configure appropriate security headers** to strengthen protection against common web-based attacks.
5. **Regularly review API security configurations** and update security controls as the API evolves.

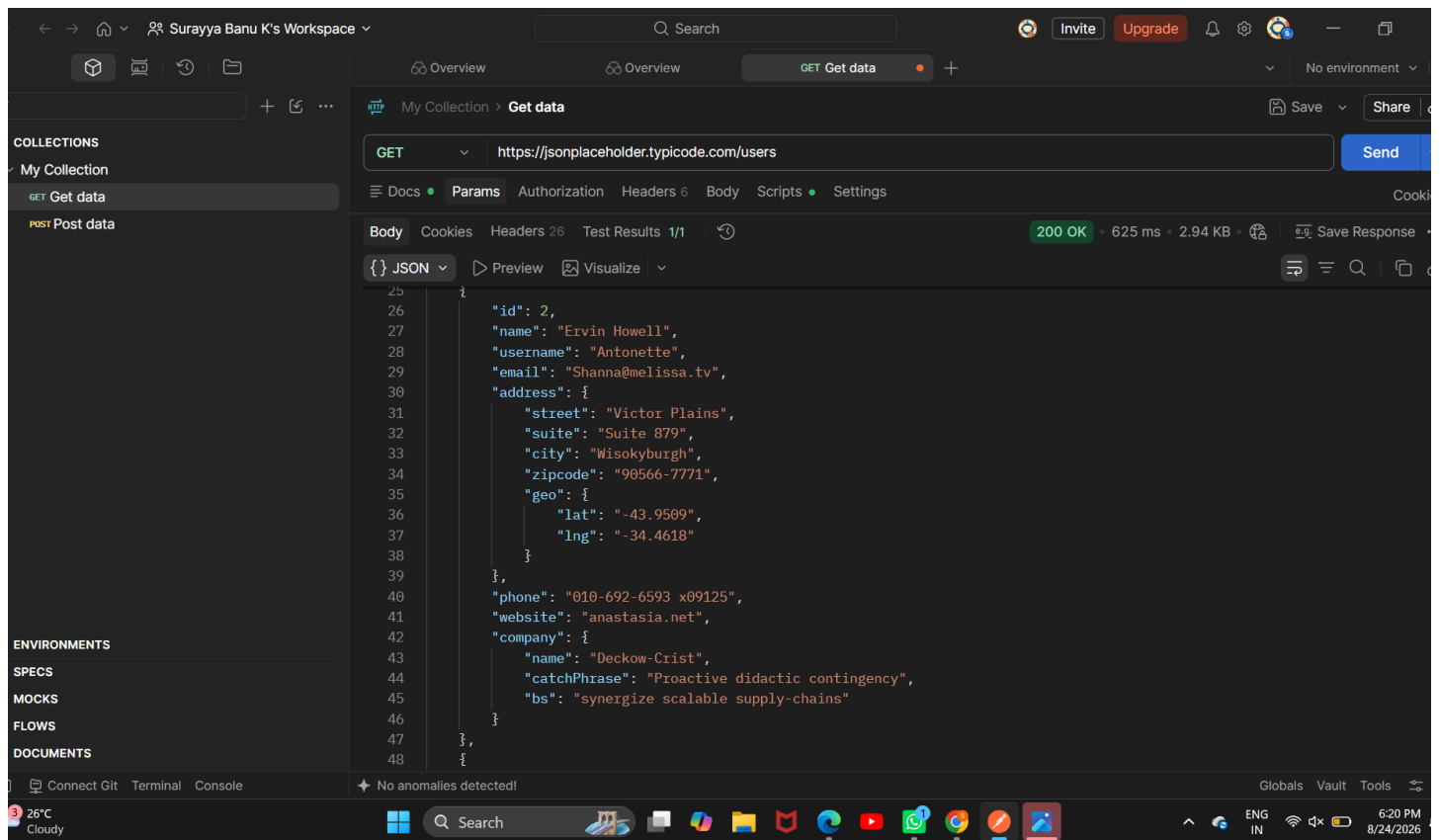
11. Screenshots / Evidence

The following evidence is taken from the user's own Postman output. The screenshots are included to show the actual requests, responses and headers used to support the observations in this report.

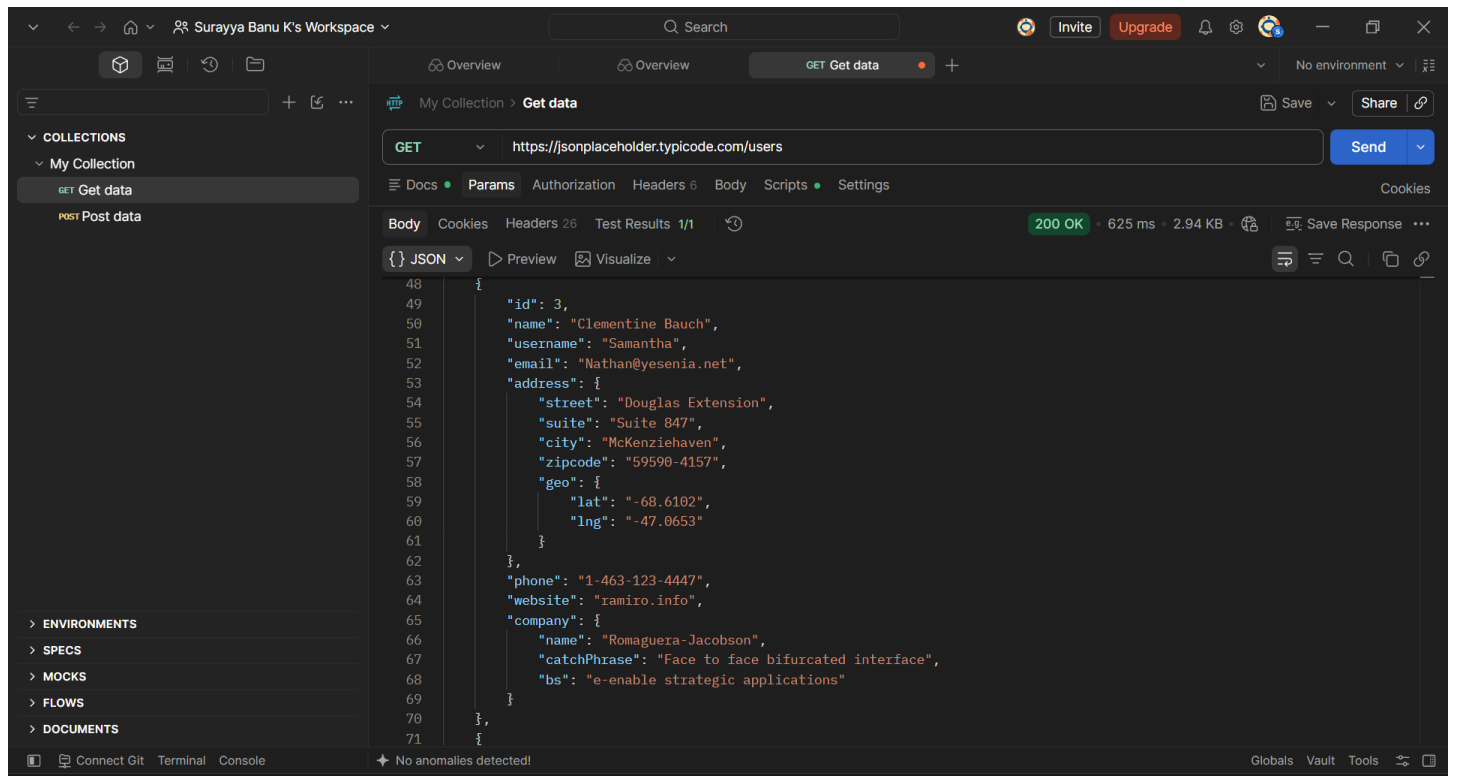
Evidence 1 — GET /users response (user record 1 and associated fields).



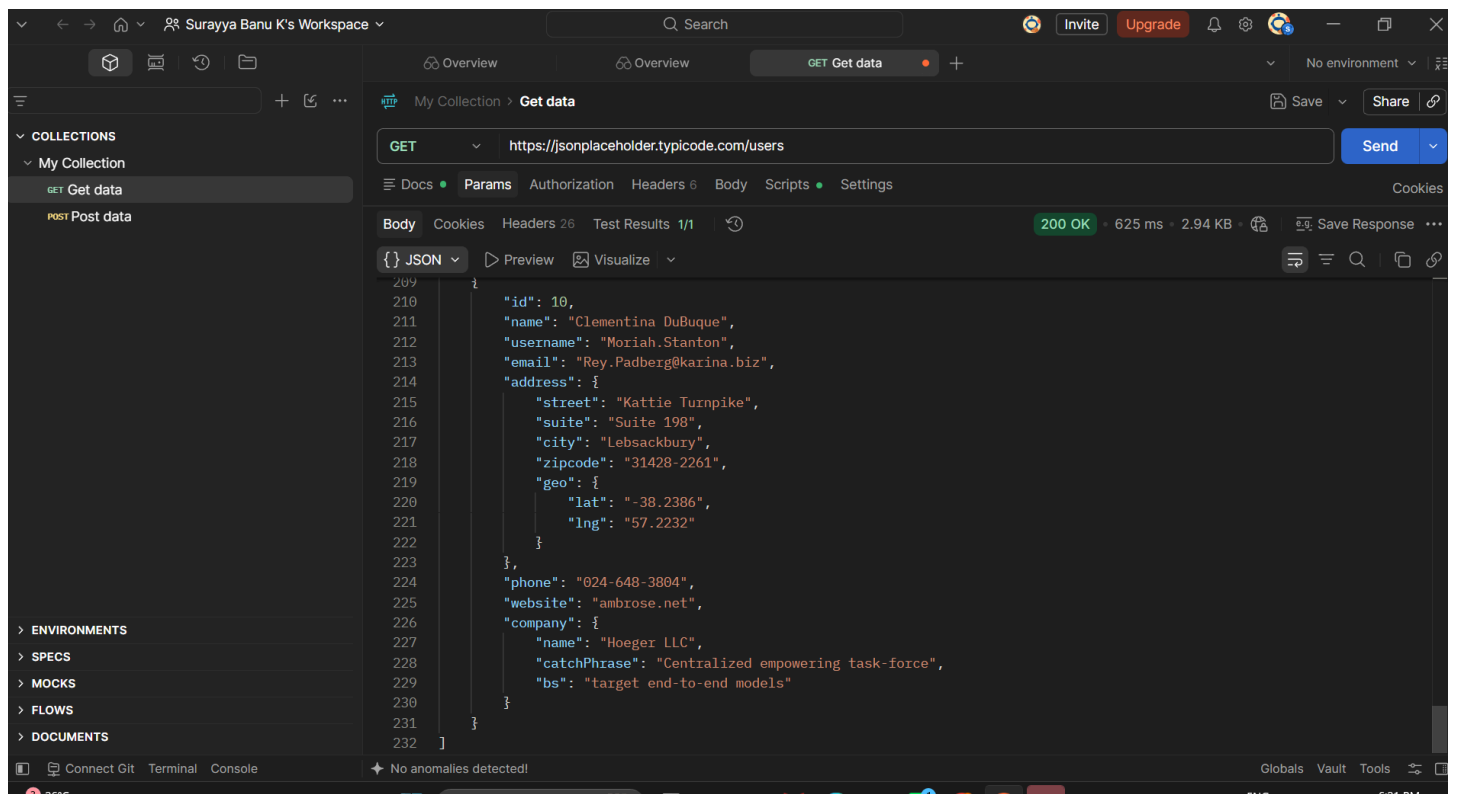
Evidence 2 — GET /users response (user record 2 and associated fields).



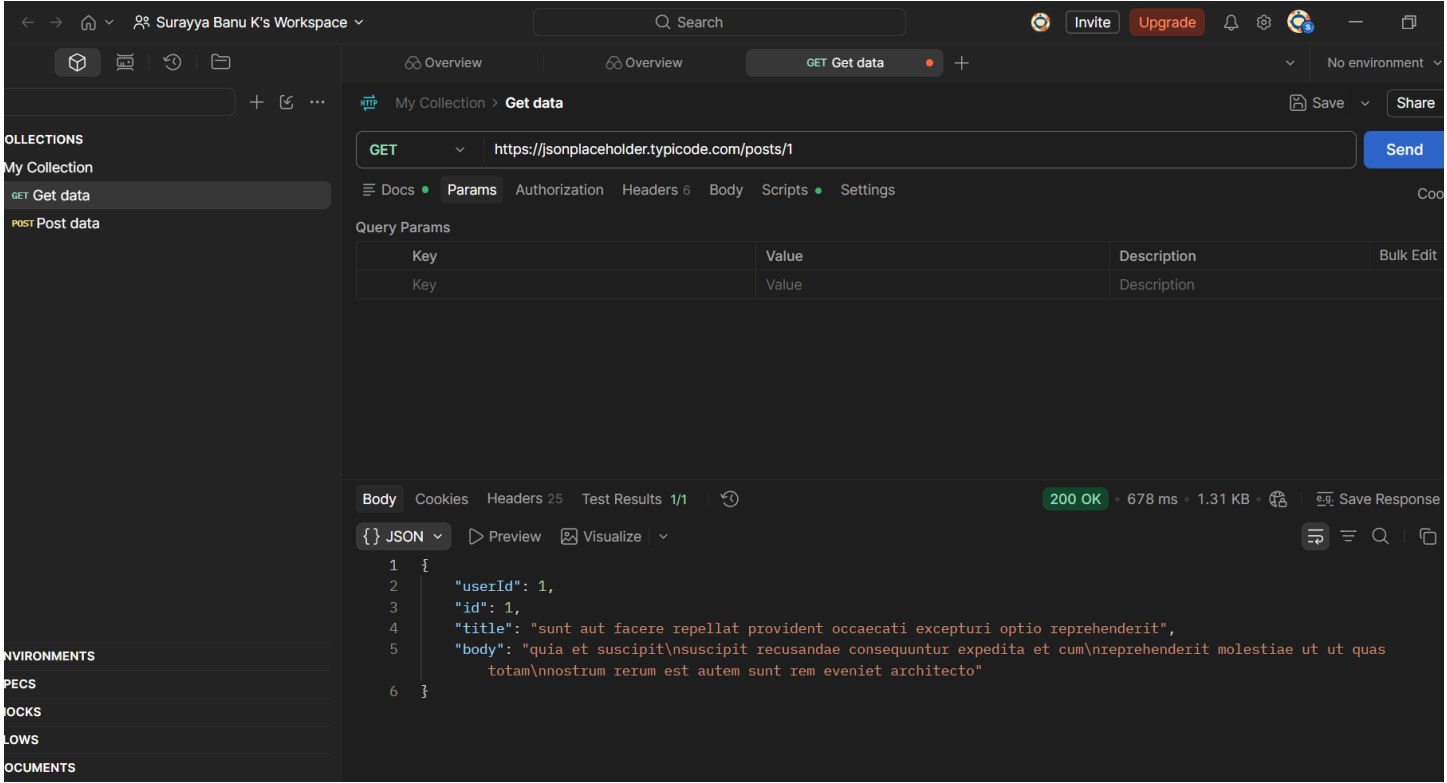
Evidence 3 — GET /users response (user record 3 and associated fields).



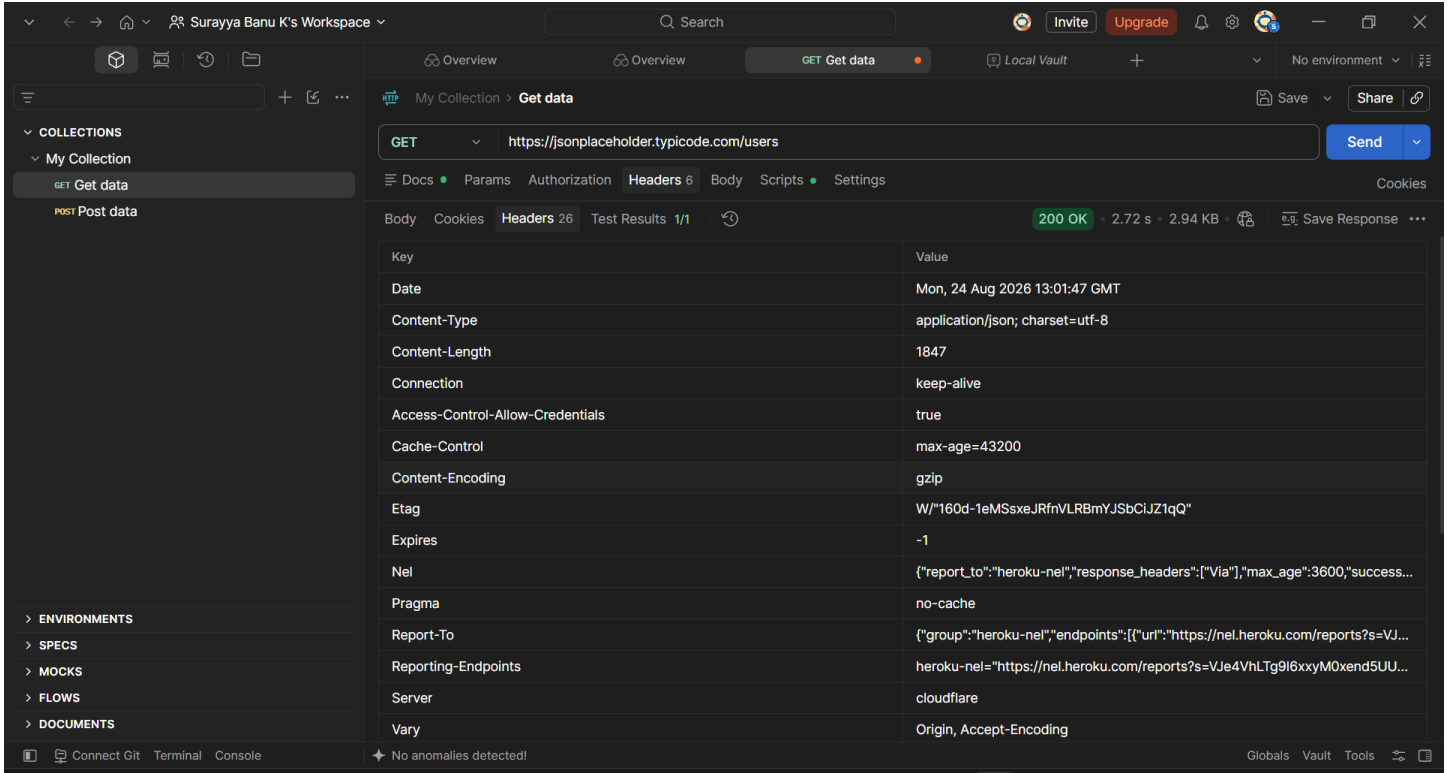
Evidence 4 — GET /users response (user record 10 and associated fields).



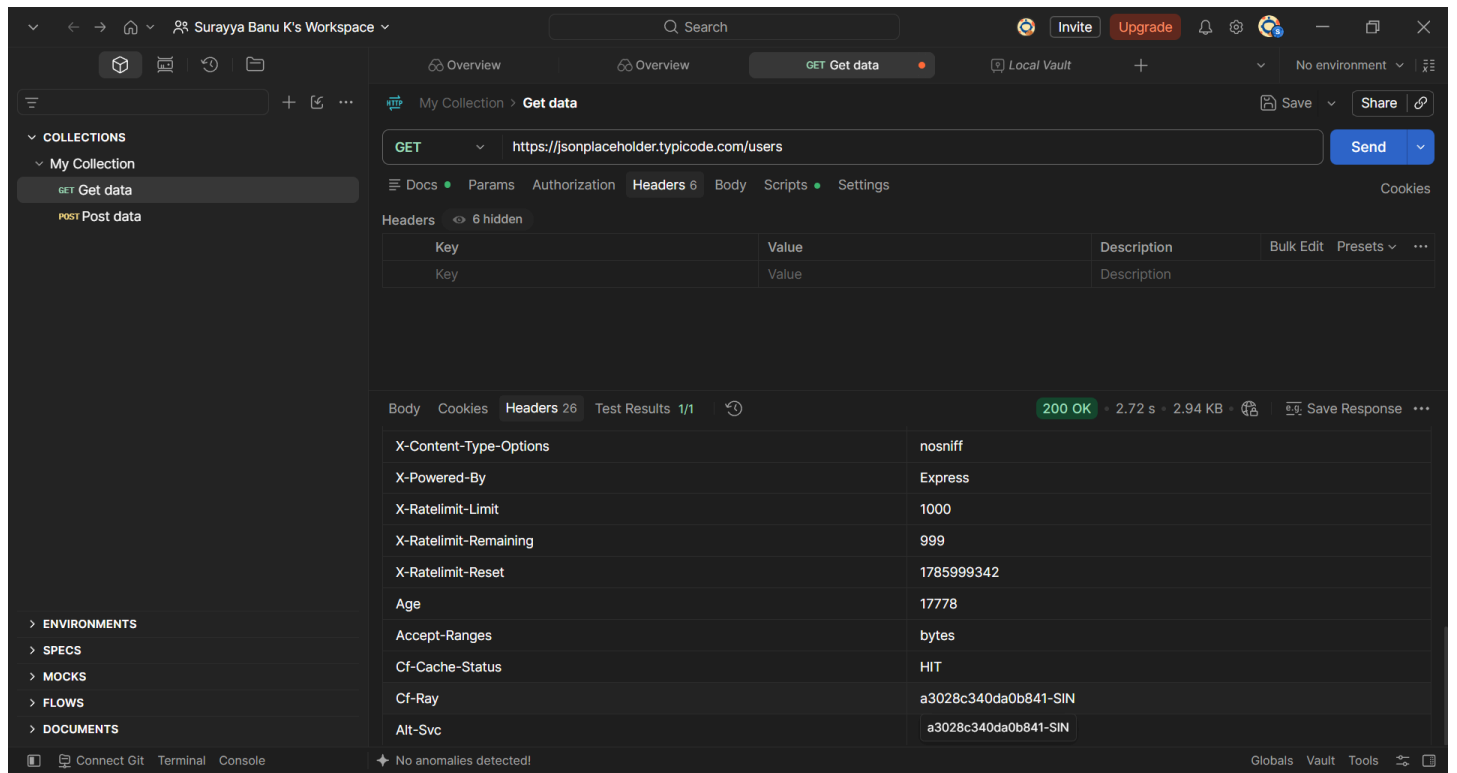
Evidence 5 — GET /posts/1 returns a JSON post object with userId, id, title and body.



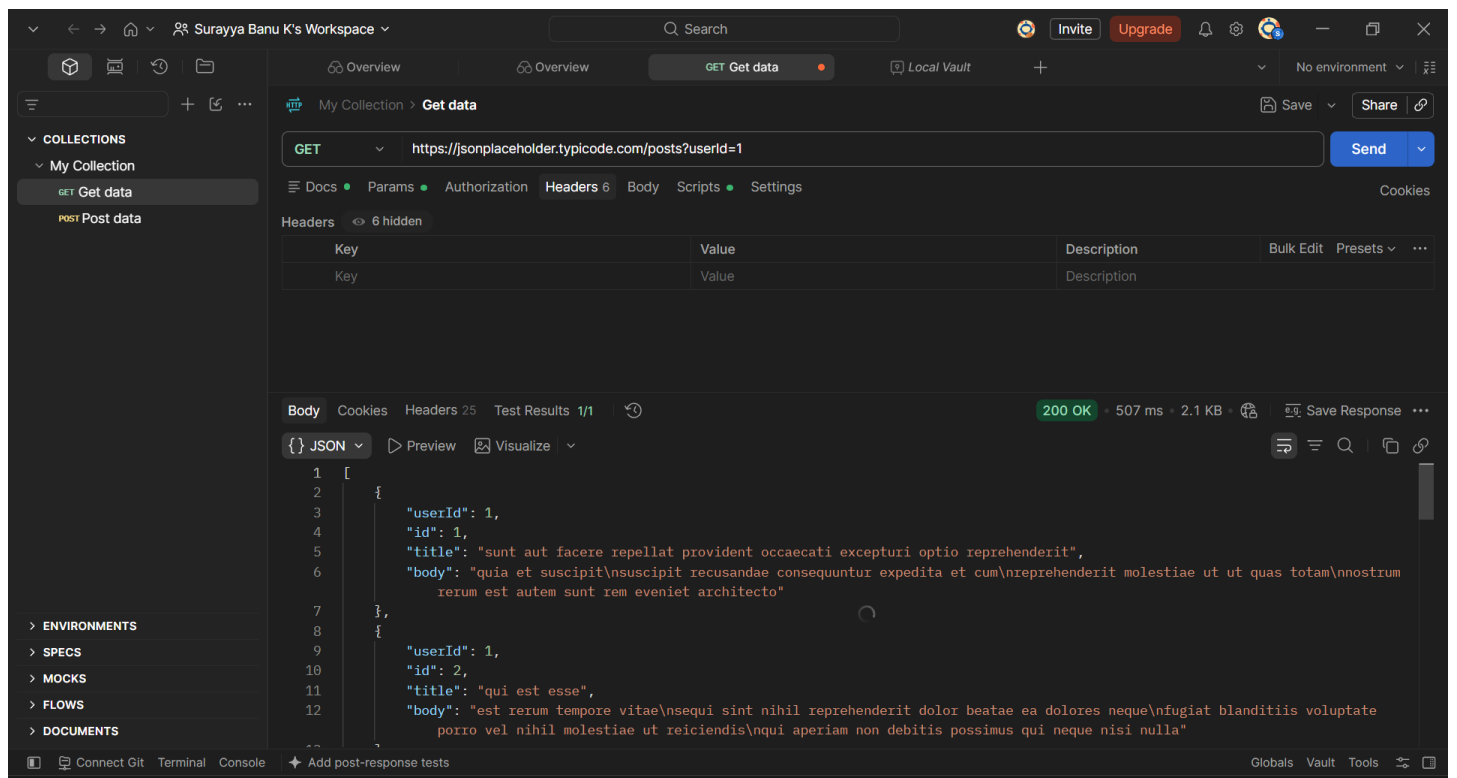
Evidence 6 — Response headers showing Content-Type, Cache-Control, server/infrastructure and other response metadata.



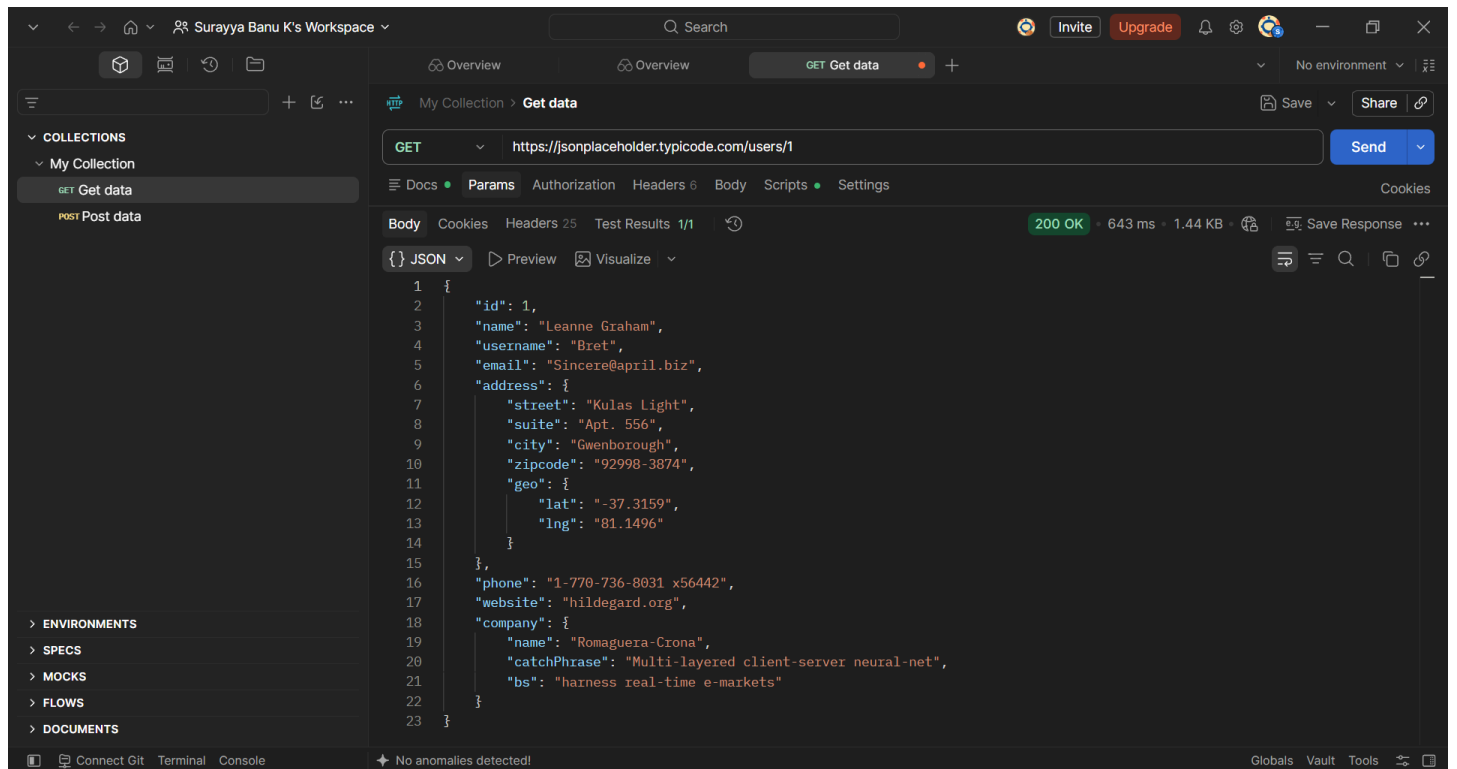
Evidence 7 — Response headers showing X-Content-Type-Options, X-Powered-By and X-RateLimit values.



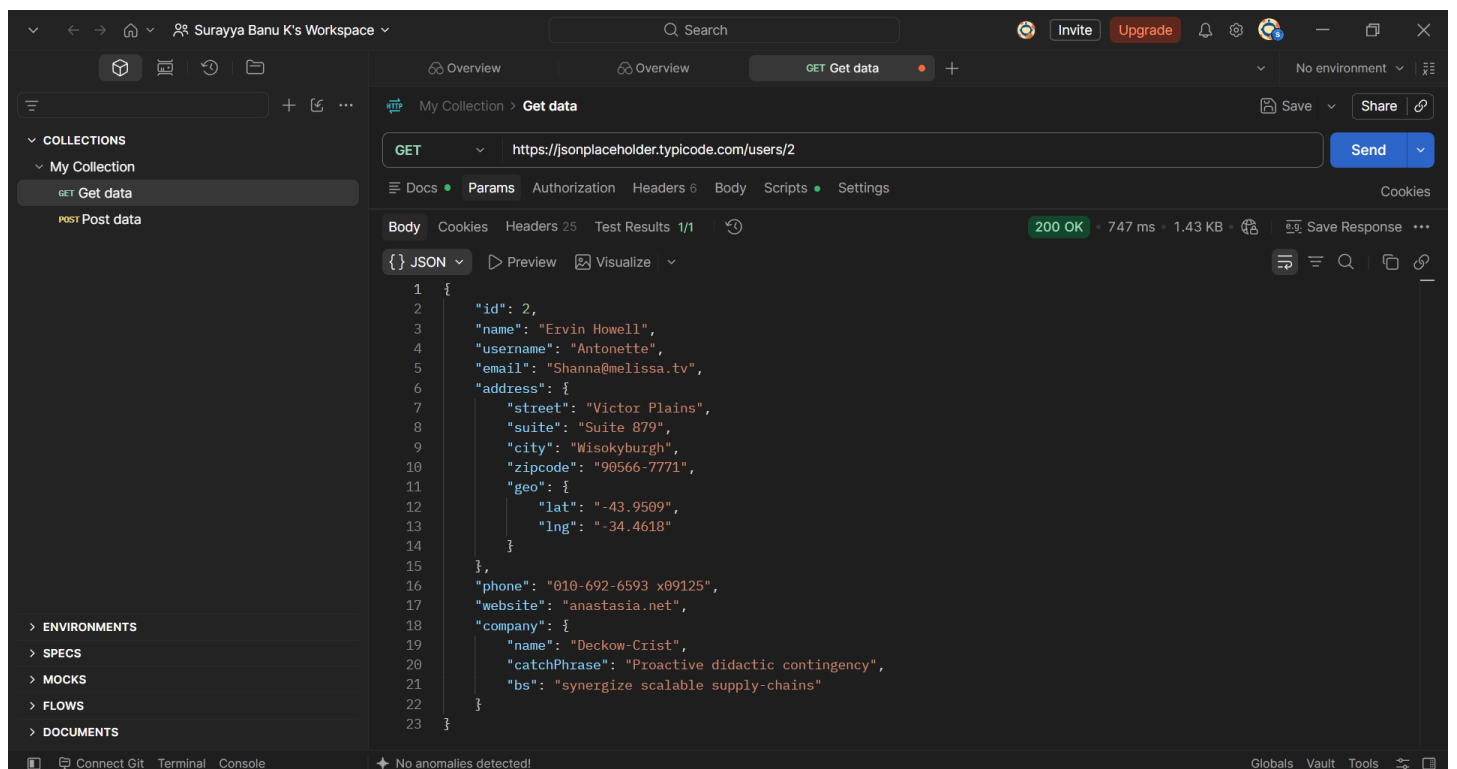
Evidence 8 — GET /posts?userId=1 query filtering returns a collection of posts for the selected userId.



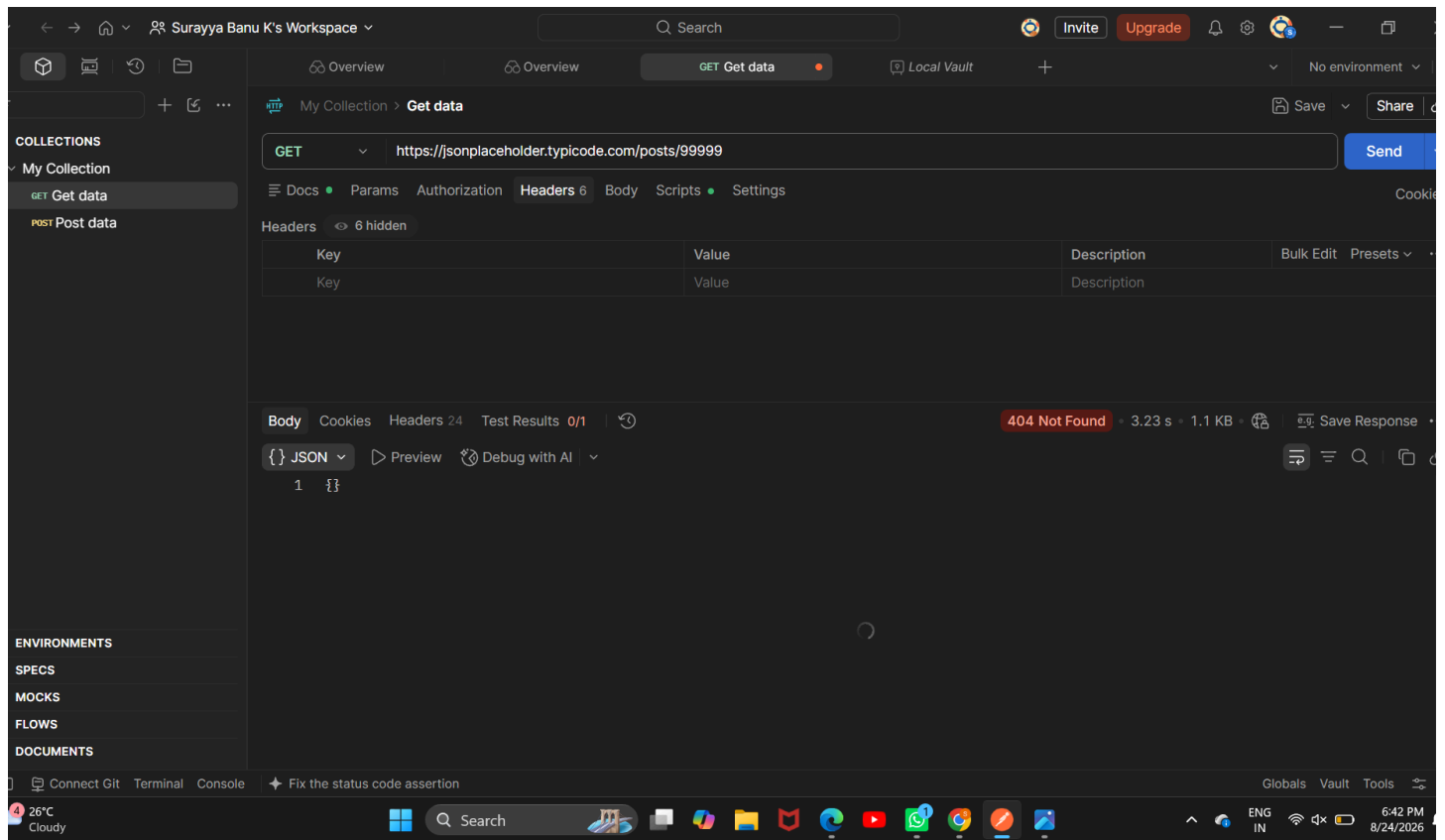
Evidence 9 — GET /users/1 returns the individual user object successfully with 200 OK.



Evidence 10 — GET /users/2 returns a different individual user object successfully with 200 OK.



Evidence 11 — GET /posts/99999 returns 404 Not Found with an empty JSON object.



12. Conclusion

This API security assessment demonstrates that even publicly accessible APIs can have security risks when proper security controls are not implemented. The assessment identified issues related to authentication, data exposure, technology information disclosure, and incomplete security headers, highlighting the importance of secure API configuration.

Proper authentication, controlled data exposure, removal of unnecessary technology information, and appropriate security headers should be implemented to strengthen API security. Overall, the assessment helped identify potential weaknesses and understand the importance of applying security controls to protect APIs and the data they handle.